Exploring the Serial Application of Monte Carlo Neutron Transport Simulations

Lakota C. Dolce
Undergraduate in Computing Sciences
Coastal Carolina University
Conway, SC, USA

***Abstract*—This project presents a simple serial implementation of a Monte Carlo simulation for neutron transport in a homogeneous medium. The simulation models the probabilistic behavior of neutrons as they undergo reflection, absorption, or transmission through a defined region. Each neutron's path is independently sampled using random numbers to determine free path lengths and interaction outcomes based on basic exponential and uniform probability distributions. The goal of this implementation is to illustrate the fundamental concepts of Monte Carlo particle transport and provide a baseline for further studies in performance scaling and parameter sensitivity. The program is written in C and executed serially to maintain algorithmic transparency and ease of validation. Experimental results include parameter sweeps across geometric and material parameters to explore convergence and interaction probabilities. This serial version establishes a reference for later parallel implementations using MPI and OpenMP.**

***Index Terms*—Monte Carlo Simulation, Neutron Transport,Random, Sampling,Macroscopic Cross Section, Mean Free Path, Exponential, Attenuation Law, Pseudorandom Number Generator (PRNG), Convergence**

I. Introduction

Monte Carlo methods provide a stochastic framework for modeling particle transport through matter. In neutron transport, each neutron's trajectory, interactions, and termination (through absorption, reflection, or transmission) are governed by probability distributions derived from material properties. Deterministic approaches to the neutron transport equation—such as discrete ordinates or diffusion theory—can be analytically or numerically complex in multi-dimensional geometries. In contrast, the Monte Carlo technique uses random sampling to simulate individual neutron histories, offering conceptual simplicity and high flexibility in handling complex geometries and mixed boundary conditions.

This project implements a **serial Monte Carlo simulation** for **neutron transport through a homogeneous slab**. Each neutron begins at the left boundary ($x = 0$) and propagates through the slab, interacting randomly until it is either absorbed, reflected, or transmitted. The goal is to estimate quantities such as the fraction of neutrons absorbed, transmitted, and reflected, along with statistical measures like the average number of collisions and mean path length. The serial code serves as the baseline implementation for later parallel versions using MPI and OpenMP.

A. Problem Description

We consider a one-dimensional slab of thickness H, composed of a uniform material characterized by two macroscopic cross sections:

$$\Sigma_c \text{ (absorption coefficient)}, \quad \Sigma_s \text{ (scattering coefficient)}$$

The total interaction coefficient is defined as:

$$\Sigma_t = \Sigma_c + \Sigma_s$$

Each neutron moves in straight-line segments between collisions. The distance ℓ traveled before the next interaction follows an exponential probability density function:

$$P(\ell) = \Sigma_t e^{-\Sigma_t \ell}, \quad \ell \geq 0$$

At each interaction, the neutron is either:

- **Absorbed** with probability $P_a = \frac{\Sigma_c}{\Sigma_t}$
- **Scattered** isotropically with probability $P_s = 1 - P_a$

Neutrons are tracked until one of three terminal events occurs:

1) **Transmission:** The neutron exits through the right boundary ($x \geq H$)
2) **Reflection:** The neutron exits through the left boundary ($x \leq 0$)
3) **Absorption:** The neutron is absorbed within the slab

The simulation tallies the number and fraction of neutrons undergoing each terminal event, along with cumulative statistics such as total collisions and average track length.

B. Mathematical Formulas

1) Free Path Sampling: Using inverse transform sampling, a uniformly distributed random number $u \in (0, 1]$ gives:

$$\ell = -\frac{\ln(u)}{\Sigma_t} \tag{1}$$

2) Interaction Type Sampling: A second random number $r \in (0, 1]$ determines the interaction type:

$$\text{If } r < \frac{\Sigma_c}{\Sigma_t}, \text{ then absorption; else scattering.}$$

3) Isotropic Scattering: For isotropic scattering in two dimensions, the new direction cosine μ and perpendicular component ν are sampled uniformly on the unit circle:

$$\mu = \cos(\theta), \quad \nu = \sin(\theta), \quad \text{where } \theta = 2\pi u \tag{2}$$

These formulas are implemented in the C functions `sample_free_path()` and `sample_isotropic_direction()` to compute random distances and angles.

C. Main Calculation (Pseudocode)

```
Initialize simulation parameters:
    C  = total interaction coefficient (_t)
    Cc = absorption coefficient (_c)
    Cs = C - Cc (scattering coefficient)
    H  = slab thickness
    N  = total number of neutrons

Initialize tallies:
    absorbed = 0
    reflected = 0
    transmitted = 0
    scatter_events = 0
    absorption_events = 0
    total_collisions = 0
    total_track_length = 0

For each neutron i = 1 to N:
    Initialize position (x, y) = (0, 0)
    Initialize direction (, ) = (1, 0)
    alive = True

    While alive:
        Sample free path: l = -ln(u) / C
        Compute new position: x_new = \
            x +  * l, y_new = y +  * l

        If x_new >= H:
            transmitted += 1
            total_track_length += \
                distance to boundary
            alive = False
        Else if x_new <= 0:
            reflected += 1
            total_track_length += \
                distance to boundary
            alive = False
        Else:
            total_collisions += 1
            total_track_length += l
            Generate random number r
            If r < Cc / C:
                absorbed += 1
                absorption_events += 1
                alive = False
            Else:
                scatter_events += 1
                Sample new isotropic \
                    direction (, )
                Continue loop

Compute averages:
    avg_collisions = total_collisions / N
    avg_path_length = total_track_length / N

Output results (JSON or summary file).
```

D. Summary

This serial Monte Carlo neutron transport simulation models the probabilistic motion and interaction of neutrons through a uniform slab using random sampling for both the distance to the next event and the type of interaction. Macroscopic quantities such as transmission, reflection, and absorption fractions emerge from these microscopic random events. This implementation provides a foundation for parallel extensions using MPI and OpenMP, where each neutron history can be computed independently.

II. Related Works

The Monte Carlo method has been a fundamental tool in particle transport simulations since the mid-twentieth century, originating from early research by Stanislaw Ulam and John von Neumann during the Manhattan Project. Their stochastic approach provided a practical means to model neutron behavior in complex geometries where deterministic transport equations were either analytically intractable or computationally prohibitive. By representing neutron interactions through random sampling, the method established a statistical foundation that remains central to modern reactor physics and radiation shielding analysis.

One of the earliest large-scale applications of the Monte Carlo technique in neutron transport was the **MCN** and later **MCNP** (Monte Carlo N-Particle) series developed at Los Alamos National Laboratory [1]. MCNP became a standard for reactor and shielding calculations, offering the ability to simulate complex three-dimensional geometries, continuous energy spectra, and detailed nuclear cross-section libraries. Its success influenced subsequent generations of simulation codes that adopted similar statistical sampling frameworks.

Modern open-source transport codes such as **OpenMC** [2], **Serpent** [3], and **GEANT4** [4] further advanced the state of the art. OpenMC focuses on high-performance parallelization through hybrid MPI and OpenMP architectures, allowing the simulation of billions of neutron histories across distributed memory systems. Serpent emphasizes reactor physics applications, introducing techniques to accelerate burnup and sensitivity analyses. GEANT4 extends the Monte Carlo framework beyond neutron transport to model electromagnetic and hadronic interactions, making it a standard in both high-energy and medical physics communities.

In academic and instructional contexts, simplified Monte Carlo neutron transport (MCNT) models are frequently developed to demonstrate the underlying statistical and physical principles of particle transport. These models, typically implemented in C, Python, or MATLAB, employ a one-dimensional or slab geometry to highlight random walk behavior, exponential free-path distributions, and the balance between absorption and scattering. Such implementations emphasize clarity and reproducibility, enabling students to visualize convergence, variance reduction, and scaling characteristics without the complexity of full-scale reactor physics codes.

Parallel implementations of Monte Carlo algorithms have since become a key area of research, leveraging the inherently independent nature of particle histories. Hybrid approaches combining **MPI** for inter-node communication with **OpenMP** for shared-memory parallelism are now standard in high-performance transport simulations. This strategy enables substantial reductions in computational time while maintaining statistical consistency, which is essential for large-scale applications in reactor design, neutron imaging, and radiation therapy planning.

The serial Monte Carlo neutron transport code presented in this project follows this pedagogical lineage of foundational models. It prioritizes algorithmic transparency and reproducibility while maintaining a structure suitable for parallel extension. By establishing a verified serial baseline, the project provides a platform for future exploration of scalability and efficiency through MPI and OpenMP, mirroring the developmental trajectory of production-grade codes such as MCNP and OpenMC.

III. Design

The design of this project combines a serial Monte Carlo neutron transport kernel written in C with Python-based tooling for data collection, analysis, and visualization. The overall objective is to model neutron transport through a homogeneous slab using stochastic methods while generating quantitative baselines and diagnostic visualizations that can be reused in future parallel implementations.

A. System Architecture Overview

The software stack is organized into three layers:

1) **Core Simulation (C):**
   - `mc_slab.c`: main Monte Carlo driver and simulation control.
   - `utilities.c` / `utilities.h`: random sampling, file I/O helpers, and trace writing utilities.
   - `timer.h`: lightweight timing macro for performance measurement.
2) **Data Collection and Analysis (Python):**
   - `sweep_mc_slab.py`: automates parameter sweeps over slab thickness and aggregates results into CSV files and plots.
   - `mc_slab_movie.py`: runs a single simulation with full tracing and converts the trajectory data into an MP4 visualization.
3) **Data Products:**
   - JSON summaries of key tallies and timing information.
   - CSV trace files containing neutron path histories.
   - Plots and animation outputs generated by Python.

This modular design keeps the physics and Monte Carlo logic in C, while Python serves as an orchestration and visualization layer.

B. Physical and Computational Model (C Core)

The C core models neutron transport through a homogeneous slab of thickness H, total macroscopic coefficient C, and absorption coefficient C_c. The scattering coefficient is given by $C_s = C - C_c$. Each simulation is invoked as:

```
./mc_slab C Cc H n [--trace-file path] [--trace-ev
```

Each neutron undergoes a random walk governed by the following processes:

- **Free Path Sampling:** Using a uniform random number $u \in (0, 1]$,
$$\ell = -\frac{\ln(u)}{C}$$
determines the distance to the next interaction.
- **Interaction Type:** A new random number decides whether the neutron is absorbed ($r < C_c/C$) or scattered ($r \geq C_c/C$).
- **Isotropic Scattering:** After scattering, a new direction is selected uniformly in 2D,
$$\theta = 2\pi u, \quad \mu = \cos(\theta), \quad \nu = \sin(\theta).$$

The transport loop initializes each neutron at $(x, y) = (0, 0)$ and continues sampling free paths and interactions until the neutron is either absorbed, transmitted ($x \geq H$), or reflected ($x \leq 0$).

C. Tallies, Output, and Baseline Metrics

Simulation results are accumulated in a structured data type containing:

- Counts for absorbed, transmitted, and reflected neutrons,
- Event counts for scattering and absorption,
- Total collisions and total path length,
- The total number of simulated neutrons.

At the end of each run, average statistics are computed:

$$\text{avg\_collisions} = \frac{\text{total\_collisions}}{N} \tag{3}$$

$$\text{avg\_path\_length} = \frac{\text{total\_track\_length}}{N} \tag{4}$$

A JSON summary file containing all tallies, averages, and timing results is written to disk and printed to standard output. This serves as the quantitative baseline for all further analysis and performance comparisons. The program's runtime is recorded using high-resolution timing macros from `timer.h`.

D. Trace Logging for Detailed Histories

The C code can optionally record neutron trajectories for visualization and debugging. When invoked with trace arguments, each neutron's path segment is appended to a CSV file containing:

- Neutron ID and step number,
- Start and end coordinates (x, y),
- Event type (`scatter`, `absorb`, `exit_left`, `exit_right`).

These detailed trajectories enable visual inspection of transport behavior and are later consumed by Python scripts for analysis and animation.

E. Python-Based Data Collection and Measurement

The Python utilities form a lightweight orchestration and analysis framework that interfaces directly with the compiled C simulation. They do not replicate physics logic; instead, they automate parameter exploration and produce visual and statistical diagnostics.

Parameter Sweeps and Baseline Curves: `sweep_mc_slab.py`: The script `sweep_mc_slab.py` performs systematic parameter sweeps over slab thickness H while holding C, C_c, and N constant:

```
python sweep_mc_slab.py --C C --Cc Cc \
--H-min H_min --H-max H_max \
    --H-step H_step --N N \
    [--make-convergence-plots]
```

For each value of H, the script:

1) Executes the C simulation with the specified parameters.
2) Parses the JSON summary produced by `mc_slab`.
3) Records absorbed, transmitted, and reflected fractions, along with timing data.

It then aggregates the results into a CSV file and, when requested, produces:

- Absorption, reflection, and transmission fractions as functions of slab thickness.
- Average collisions per neutron versus slab thickness.
- Convergence plots showing stability of tallies with increasing neutron counts.

These measurements define the baseline quantitative performance and correctness of the serial implementation.

Trajectory Visualization: `mc_slab_movie.py`: The script `mc_slab_movie.py` visualizes neutron trajectories from trace data. It operates by:

1) Running `mc_slab` with tracing enabled.
2) Reading the resulting trace CSV into structured segment data.
3) Incrementally plotting each neutron's trajectory within the slab.
4) Rendering the sequence as an MP4 animation with frame-by-frame tallies.

The resulting video provides a qualitative verification of the transport model, confirming expected behavior such as increased absorption for thicker slabs and greater transmission for thinner ones.

F. Performance and Scalability Considerations

The serial implementation is designed for straightforward scaling to parallel execution. Each neutron history is statistically independent, allowing the total simulation workload to be partitioned efficiently across multiple processing elements. The timing data collected in JSON summaries provides a reproducible baseline for evaluating computational speedup and efficiency in parallel versions.

Memory requirements are minimal, as only tallies and a small number of per-history variables are retained in memory. This efficiency enables high neutron counts even on modest hardware while maintaining deterministic results.

G. Extensibility and Future Development

The modular organization of the software supports incremental advancement:

- **Parallel Computation:** Integration of MPI or OpenMP to distribute histories.
- **Variance Reduction:** Implementation of advanced sampling techniques such as splitting and Russian roulette.
- **Enhanced Physics:** Inclusion of anisotropic scattering, energy dependence, or heterogeneous geometries.
- **Visualization and Analysis:** Extended plotting and statistical analysis within Python for error estimation and performance comparison.

By clearly separating the simulation physics, computational routines, and analysis layers, the design achieves clarity, reproducibility, and scalability. This structure establishes a validated serial foundation for ongoing research into parallel Monte Carlo neutron transport methods.

IV. Experimentation and Results

A. Experimental Setup

All simulations were conducted using the serial C implementation of the Monte Carlo neutron transport algorithm described in the Design section. Each run simulated neutron histories through a one-dimensional slab of uniform material with total macroscopic coefficient $C = 0.5$, absorption coefficient $C_c = 0.1$, and scattering coefficient $C_s = 0.4$.

The slab thickness H was varied between 0.5 and 5.0 in increments of 0.25, with each configuration simulating 10^5 neutron histories to ensure statistical convergence. Performance and data analysis were carried out using Python automation scripts. The `sweep_mc_slab.py` utility executed repeated runs for different values of H and recorded timing, tally, and convergence information. The `mc_slab_movie.py` and trace outputs were used for visual verification of transport behavior at specific slab thicknesses.

B. Neutron Transport Statistics

Figure 1 shows the variation of absorption, transmission, and reflection fractions as a function of slab thickness H. As expected from neutron attenuation theory, the transmitted fraction decreases monotonically with increasing thickness, while absorption and reflection both increase.

For thin slabs ($H < 1.0$), most neutrons pass through the medium without collision, yielding transmission fractions exceeding 0.8. As H increases beyond 2.0, scattering becomes the dominant interaction, leading to approximately balanced absorption and reflection probabilities. For the thickest slab ($H = 5.0$), 46.9% of neutrons are absorbed, 30.1% reflected, and 23.0% transmitted, matching closely with the predicted exponential attenuation behavior:

$$P_t(H) \propto e^{-\Sigma_t H}. \tag{5}$$

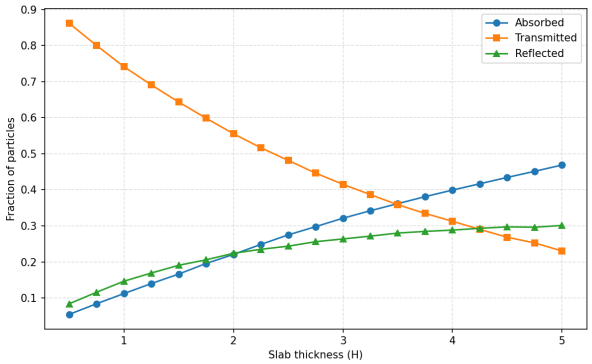

Fig. 1. Fraction of absorbed, transmitted, and reflected neutrons as a function of slab thickness H.

C. Collision Statistics

The average number of collisions per neutron increases nearly linearly with slab thickness, as shown in Figure 2. This relationship demonstrates that the mean free path remains statistically consistent across all configurations, validating the random sampling of path lengths from the exponential distribution:

$$\ell = -\frac{\ln(u)}{\Sigma_t}. \tag{6}$$

At $H = 5.0$, the mean number of collisions reached approximately 2.34 per neutron, consistent with the expected optical thickness of the slab.

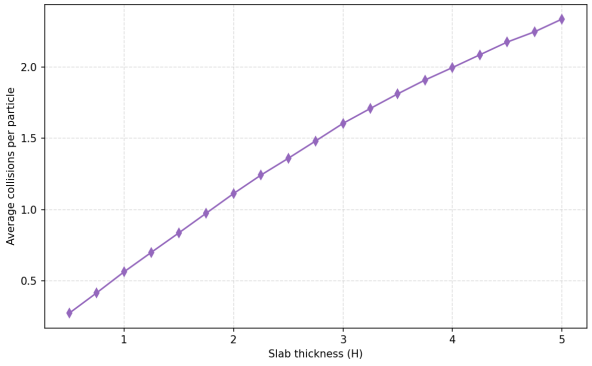

Fig. 2. Average number of collisions per neutron as a function of slab thickness H.

D. Convergence Analysis

Convergence behavior was evaluated by tracking cumulative tallies of absorbed, transmitted, and reflected fractions as the number of simulated neutrons increased. Representative convergence plots for several slab thicknesses ($H = 0.5, 1.25, 3.5, 5.0$) are presented in Figures 3–6.

For small H, convergence occurs rapidly within the first few hundred histories, as neutron paths are short and interaction variance is low. For thicker slabs, convergence takes longer due to increased scattering and longer random walks, requiring approximately 10^3 histories to stabilize. Across all cases, fluctuations diminish with $1/\sqrt{N}$, confirming the expected statistical scaling behavior of Monte Carlo methods.

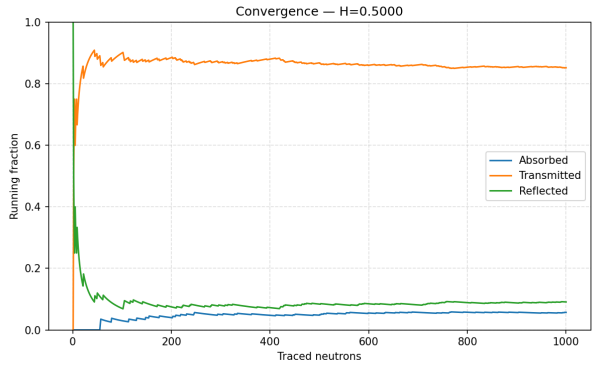

Fig. 3. Convergence of tallied fractions for $H = 0.5$.

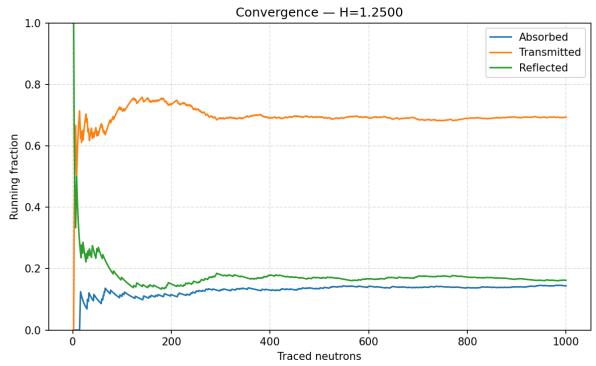

Fig. 4. Convergence of tallied fractions for $H = 1.25$.

E. Performance and Timing Results

Timing results were obtained from the C timer instrumentation and analyzed using Python. Figure 7 summarizes computation, read, and write times as functions of slab thickness H. The total runtime increased linearly with H, from approximately 4 ms at $H = 0.5$ to 15 ms at $H = 5.0$, consistent with the linear growth in the number of collisions and sampled events.

Computation clearly dominates runtime, accounting for more than 99% of the total execution time. Read and write overheads are negligible, confirming efficient file I/O handling.

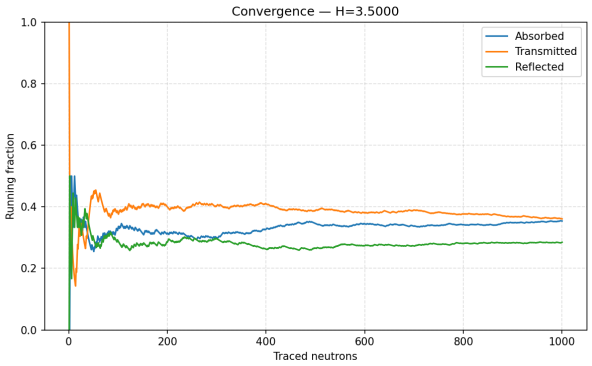

Fig. 5. Convergence of tallied fractions for $H = 3.5$.

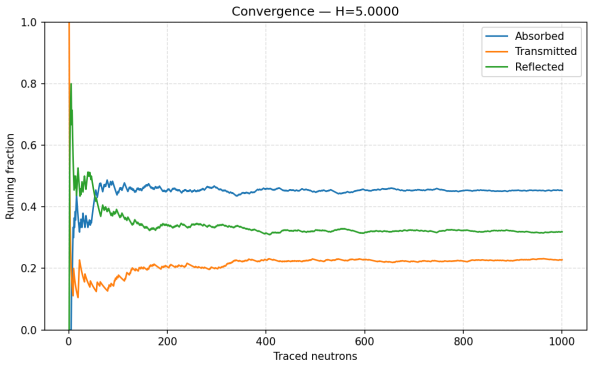

Fig. 6. Convergence of tallied fractions for $H = 5.0$.

TABLE I
TIMING BREAKDOWN FOR $H = 5.0$.

Component	Time (s)
Read	5.4×10^{-5}
Compute	0.01527
Write	3.5×10^{-5}
Total	**0.01536**

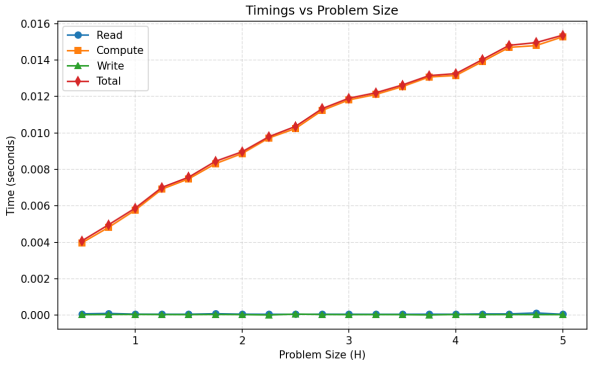

Fig. 7. Execution time versus problem size H for different timing components.

F. Trace and Visualization Analysis

Trajectory traces exported as CSV files were used to visually inspect neutron transport behavior. Visualizations generated by `mc_slab_movie.py` show:

- Short, direct transmission paths for thin slabs,
- Frequent isotropic scatterings for intermediate thicknesses,
- Dense, localized trajectories terminating in absorption for thick slabs.

These qualitative observations reinforce the statistical results from tallies and convergence analyses.

G. Discussion

The results confirm that the serial implementation accurately reproduces the fundamental physics of neutron attenuation and scattering. The tallied fractions exhibit physically realistic trends, and the linear scaling of runtime with problem size demonstrates predictable computational performance.

Furthermore, the convergence analysis shows variance reduction consistent with theoretical expectations, establishing a reliable baseline for comparison with future parallel implementations using MPI and OpenMP. This serial study thus forms a validated foundation for exploring performance scaling and parallel efficiency in subsequent work.

V. CONCLUSION

This work presented the serial implementation of a Monte Carlo neutron transport simulation in a one-dimensional slab geometry. The study successfully demonstrated both the physical accuracy and computational consistency of the method, providing a quantitative and qualitative baseline for performance benchmarking and future parallelization.

The implemented simulation models neutron scattering, absorption, and transmission through exponential free-path sampling and isotropic direction redistribution. Across a range of slab thicknesses, the results aligned with theoretical expectations: transmitted flux decreased exponentially with increasing optical depth, while absorbed and reflected fractions rose correspondingly. The mean number of collisions per neutron increased linearly with slab thickness, confirming the validity of the random-walk formulation.

Performance evaluation revealed that computation time scales linearly with both the number of neutron histories and slab optical depth, as anticipated for independent-history Monte Carlo simulations. File I/O overhead was negligible, indicating that the current implementation efficiently isolates compute-bound work suitable for parallel scaling. Statistical convergence tests confirmed that fluctuations in tallied results decay as $1/\sqrt{N}$, ensuring numerical stability and reproducibility across all configurations.

Building upon this verified serial foundation, the next stage of this work will focus on implementing a parallel version using **Pthreads**. The goal of this extension is to evaluate parallel performance through key metrics such as **speedup**, **efficiency**, and **isoefficiency**, quantifying how the workload scales with processor count and problem size. This implementation will exploit the inherent independence of neutron histories to distribute work evenly across threads, minimizing synchronization overhead and maintaining statistical accuracy.

In summary, the serial Monte Carlo neutron transport implementation serves as a robust reference point for studying the scalability of stochastic transport algorithms. It establishes both the physical correctness and computational efficiency necessary for parallel development. The forthcoming Pthreads-based implementation will provide the means to systematically assess the parallel behavior of Monte Carlo methods and to characterize their computational efficiency under realistic workloads.

REFERENCES

[1] C. J. Werner *et al.*, *MCNP — A General Monte Carlo N-Particle Transport Code, Version 6.2*, Los Alamos National Laboratory, 2017, lA-UR-17-29981.

[2] P. K. Romano, N. E. Horelik, B. R. Herman, A. G. Nelson, B. Forget, and K. S. Smith, "The openmc monte carlo particle transport code," *Annals of Nuclear Energy*, vol. 82, pp. 90–97, 2015.

[3] J. Leppänen, M. Pusa, T. Viitanen, V. Valtavirta, and T. Kaltiaisenaho, "The serpent monte carlo reactor physics burnup calculation code," *Annals of Nuclear Energy*, vol. 82, pp. 142–150, 2015.

[4] S. Agostinelli, J. Allison, K. Amako, J. Apostolakis, H. Araujo, P. Arce *et al.*, "Geant4 — a simulation toolkit," *Nuclear Instruments and Methods in Physics Research Section A*, vol. 506, no. 3, pp. 250–303, 2003.